El código de ML, paso a paso

TI3002C – Etapa 5.1 · Jaime Hernández González (A00840312) · Equipo 6 · Explicación de cada sección del notebook con su código y sus resultados

> **La idea en una frase:** le enseñamos a la computadora cómo se vendió cada producto en el pasado, para que adivine cuánto se va a vender el mes que sigue.
> **Los 4 bloques:** (1) preparar los datos → secciones 1–5 · (2) entrenar modelos → 6–10 · (3) revisar qué tan bien adivinan → 11–13 · (4) concluir → 14.

Sección 1 · Librerías

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pd.set_option('display.max_colwidth', None)
```

Línea	Qué hace
`pandas as pd`	Trabaja con tablas (como Excel). `pd` es su apodo.
`numpy as np`	Cálculos (valor absoluto, raíz cuadrada…).
`matplotlib` / `seaborn`	Gráficas.
`RandomForestRegressor`	Modelo 1.
`XGBRegressor`	Modelo 2.
`mean_absolute_error`, etc.	Fórmulas ya hechas para medir el error.
`set_option(...)`	Que las tablas muestren el texto completo, sin "...".

Sección 2 · Importar datos y exploración

```
df = pd.read_csv('data_DanuStore_Equipo6_Mensual.csv')
print(df.shape)
df.head()
```

```
(21840, 20)
```

- `read_csv` lee el archivo y lo guarda en la tabla **`df`**.
- **Cada fila = un producto, en una región, en un mes.** 130 productos × 7 regiones × 24 meses = 21,840 filas.
- En Colab hay que cambiar la ruta del archivo (las líneas con `#` son para conectar Google Drive).

2.1 Exploración inicial

```
print('Valores nulos en toda la base:', df.isnull().sum().sum())
print('Registros duplicados:', df.duplicated().sum())
print('Productos:', df['product_id'].nunique(), '| Regiones:', df['region'].nunique(), '| Categorías:',
df['category'].nunique())
df[['units_sold', 'units_expected', 'stock', 'static_price']].describe().round(2)
```

```
Valores nulos en toda la base: 0
Registros duplicados: 0
Productos: 130 | Regiones: 7 | Categorías: 4
```

- `isnull()` : ¿hay celdas vacías? `duplicated()` : ¿filas repetidas? `nunique()` : ¿cuántos valores distintos?
- `describe()` : promedio, mínimo, máximo y cuartiles de cada columna.
- Después hay 2 gráficas: un **histograma** (la mayoría de las filas vende poco y pocas venden mucho: sesgo a la derecha) y las **ventas por región** (`groupby('region').sum()` : CDMX, Monterrey y Guadalajara venden más).

2.2 Preparar la fecha

```
meses = {'January': 1, 'February': 2, 'March': 3, ... , 'December': 12}

df['mes'] = df['month'].map(meses)
df['fecha'] = pd.to_datetime(dict(year=df['year'], month=df['mes'], day=1))

df = df.sort_values(['product_id', 'region', 'fecha']).reset_index(drop=True)
```

Línea	Qué hace	Ejemplo
`meses = {...}`	Diccionario que traduce nombre → número	'March' → 3
`.map(meses)`	Aplica la traducción a toda la columna	March → 3
`pd.to_datetime(...)`	Junta año + mes + día 1 en una fecha	2025, 3 → 2025-03-01
`sort_values([...])`	Ordena por producto, región y fecha	ene 2024, feb 2024…

Ordenar es clave: el siguiente paso usa "el renglón de arriba", y eso solo funciona si los meses están en orden.

2.3 Ventas a través del tiempo

```
por_mes = df.groupby('fecha')[['units_sold', 'units_expected']].sum()

plt.plot(por_mes.index, por_mes['units_sold'], marker='o', label='Ventas reales (units_sold)')
plt.plot(por_mes.index, por_mes['units_expected'], marker='o', label='Ventas esperadas
(units_expected)')
plt.show()

print('Se vendió el', round(por_mes['units_sold'].sum() / por_mes['units_expected'].sum() * 100, 1), '%
de lo esperado')
```

```
Se vendió el 17.0 % de lo esperado
```

- `groupby('fecha').sum()` : suma las ventas de todos los productos y regiones en cada mes.
- Gráfica: lo **esperado** (naranja) es plano y altísimo; lo **real** (azul) es mucho menor y sube en noviembre–diciembre.
- La segunda gráfica usa `pivot_table` para hacer una tabla con una fila por mes y una columna por región, y dibuja una línea por región.

2.4 Comportamiento por categoría

```
por_categoria_mes = df.pivot_table(index='fecha', columns='category', values='units_sold',
aggfunc='sum')
por_categoria_mes.plot(figsize=(11, 4), marker='.')

por_categoria = df.groupby('category')[['units_sold', 'units_expected']].sum()
por_categoria['% de las ventas totales'] = (por_categoria['units_sold'] /
por_categoria['units_sold'].sum() * 100).round(1)
por_categoria['% vendido de lo esperado'] = (por_categoria['units_sold'] /
por_categoria['units_expected'] * 100).round(1)
```

Categoría	% de las ventas	% vendido de lo esperado
Casual Wear	28.7 %	18.0 %
Footwear	26.3 %	15.4 %
Fitness Wear	25.0 %	18.0 %
Accessories	20.0 %	16.8 %

Las 4 categorías tienen la misma temporada (suben ~21–25 % en nov–dic y bajan en febrero). Esto responde al "beneficio para el cliente": conocer cómo se comportan sus productos.

Sección 3 · Selección de variables

3.1 Crear las "ventas pasadas" (lo más importante)

```
grupo = df.groupby(['product_id', 'region'])['units_sold']

df['ventas_mes_anterior'] = grupo.shift(1)
df['ventas_hace_2'] = grupo.shift(2)
df['ventas_hace_3'] = grupo.shift(3)
df['promedio_3_meses'] = (df['ventas_mes_anterior'] + df['ventas_hace_2'] + df['ventas_hace_3']) / 3
df['ventas_hace_12'] = grupo.shift(12)   # solo para revisarla
```

- `groupby(['product_id','region'])`: separa en "series" (playera 1 en Bajío, playera 1 en CDMX…). Son 910 series, y cada una usa **sus propias** ventas.
- `shift(1)`: recorre la columna un renglón hacia abajo dentro de cada serie.

fecha	units_sold	shift(1)	shift(2)	shift(3)	promedio_3_meses
2024-01	136	—	—	—	—
2024-02	161	136	—	—	—
2024-03	148	161	136	—	—
2024-04	161	148	161	136	**148.3**
2024-05	177	161	148	161	**156.7**

👉 En la fila de abril, el modelo "ve" enero, febrero y marzo: justo lo que conocerías para pronosticar abril.

3.2 Variables prohibidas (solo texto)

`Ventas_Acumuladas_Mensual`, `Venta_Promedio_Diaria`, `sales_amount`, `Cumplimiento_Venta_Esperada`, `Dias_Cobertura_Inventario` y `Stock_Restante_Mensual` se calculan **con la misma venta que queremos adivinar**: usarlas sería hacer trampa (fuga de información). `units_expected` no es pista: es la comparación.

3.3 Correlaciones

```
correlacion = df[variables_numericas].corr(method='spearman')
sns.heatmap(correlacion, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
print(correlacion['units_sold'].sort_values(ascending=False).round(3))
```

```
ventas_hace_12         1.000   ← idéntica (sospechoso)
promedio_3_meses       0.950   ← la mejor pista
ventas_mes_anterior    0.939
ventas_hace_3          0.927
ventas_hace_2          0.927
units_expected         0.715
mes                    0.093
static_price          -0.038
```

```
tabla = df.pivot_table(index=['product_id', 'region', 'mes'], columns='year', values='units_sold')
iguales = (tabla[2024] == tabla[2025]).mean() * 100
```

```
Registros donde las ventas de 2025 son IGUALES a las de 2024: 76.6 %
```

```
print('Productos distintos:', df['product_id'].nunique())
print('Precios distintos:', df['static_price'].nunique())
```

```
Productos distintos: 130
Precios distintos: 130
```

Variable	¿Se usa?	Por qué
Ventas pasadas (1, 2, 3 meses y promedio)	✅ Sí	Las más relacionadas (~0.93–0.95)
mes	✅ Sí	Correlación baja sola, pero captura la temporada (nov–dic)
región, categoría, subcategoría	✅ Sí	Distinguen tipos de producto y zonas
static_price	❌ No	Cada producto tiene su propio precio: serviría para "reconocer" el producto y memorizar
ventas_hace_12	❌ No (solo comparación)	2025 repite a 2024 en un 77 %; el modelo solo copiaría

Sección 4 · Codificar variables categóricas

```
datos = df.dropna(subset=['promedio_3_meses']).copy()

variables = ['region', 'category', 'subcategory', 'mes',
             'ventas_mes_anterior', 'ventas_hace_2', 'ventas_hace_3', 'promedio_3_meses']

X = pd.get_dummies(datos[variables], columns=['region', 'category', 'subcategory'], dtype=int)
y = datos['units_sold']
```

```
Registros: 19110 | Columnas después de codificar: 29
```

- `dropna`: quita los 3 primeros meses de cada serie (no tienen ventas pasadas).
- `X` **= las pistas** (lo que el modelo ve). `y` **= la respuesta** (lo que tiene que adivinar).
- `get_dummies` (One-Hot): una columna de 0/1 por cada región, categoría y subcategoría. Ej.: Bajío → region_Bajío = 1, region_CDMX = 0…

Sección 5 · Normalización

No se hace. Los árboles hacen preguntas tipo *"¿ventas_mes_anterior > 150?"*; la respuesta es la misma en piezas, en miles o estandarizado. Sí haría falta en KNN, regresión regularizada o redes neuronales.

Sección 6 · Partición de los datos (por tiempo)

```
particiones = {
    '80/20': '2025-09-01',
    '70/30': '2025-07-01',
    '60/40': '2025-05-01',
}

for nombre, corte in particiones.items():
    entrenamiento = datos['fecha'] < corte
    print(nombre, '-> entrenamiento:', entrenamiento.sum(), 'registros | prueba:',
(~entrenamiento).sum(), 'registros')
```

```
80/20 -> entrenamiento: 15470 registros | prueba: 3640 registros
70/30 -> entrenamiento: 13650 registros | prueba: 5460 registros
60/40 -> entrenamiento: 11830 registros | prueba: 7280 registros
```

- `particiones` es un diccionario: nombre de la partición → fecha de corte.
- `datos['fecha'] < corte` da True/False por fila: **True = entrenamiento** (meses viejos), **False = prueba** (meses nuevos). El `~` invierte (False→True).
- `.sum()` sobre True/False cuenta cuántos True hay.

Partición	Entrena con	Prueba con
80/20	abr 2024 – ago 2025 (17 meses)	sep – dic 2025 (4 meses)
70/30	abr 2024 – jun 2025 (15 meses)	jul – dic 2025 (6 meses)
60/40	abr 2024 – abr 2025 (13 meses)	may – dic 2025 (8 meses)

> **¿Por qué no aleatoria?** Porque queremos pronosticar el futuro. Si mezcláramos al azar, el modelo entrenaría con diciembre para "adivinar" octubre: haría trampa y los resultados saldrían inflados.
> **Ojo:** hasta la sección 12 es un pronóstico **a un mes** (para cada mes de prueba se usan las ventas reales del mes anterior). El pronóstico de varios meses va en la sección 13.

Sección 7 · Métricas y líneas base

```
def evaluar(y_real, y_pred):
    mae = mean_absolute_error(y_real, y_pred)
    rmse = np.sqrt(mean_squared_error(y_real, y_pred))
    wape = np.abs(y_real - y_pred).sum() / y_real.sum() * 100
    r2 = r2_score(y_real, y_pred)
    return {'MAE': round(float(mae), 2), 'RMSE': round(float(rmse), 2), 'WAPE (%)': round(float(wape),
2), 'R2': round(float(r2), 3)}
```

`def` crea una **función**: una "receta" que recibe lo real y lo pronosticado y regresa las 4 métricas en un diccionario. Así no repetimos estas líneas 30 veces.

Métrica	Fórmula en palabras	Cómo se lee
MAE	Promedio de |real − pronóstico|	"En promedio me equivoco por 13 unidades"
RMSE	Raíz del promedio de los errores al cuadrado	Como el MAE, pero castiga más los errores grandes
WAPE	Suma de errores ÷ suma de ventas × 100	"Me equivoco en 6 de cada 100 piezas vendidas" (la principal)
R^2	Qué parte de la variación explica	1 = perfecto, 0 = no explica nada

7.1 Líneas base (contra qué comparamos)

```
resultados = []

for nombre, corte in particiones.items():
    prueba = datos['fecha'] >= corte
    fila = evaluar(y[prueba], datos.loc[prueba, 'units_expected'])
    fila['Modelo'] = 'Pronóstico actual (units_expected)'
    fila['Partición'] = nombre
    fila['Hiperparámetros'] = '-'
    resultados.append(fila)
    ... (igual con 'ventas_mes_anterior' y 'ventas_hace_12')
```

- `resultados = []`: una lista vacía donde iremos guardando una "fila" por cada prueba.
- `y[prueba]`: las ventas reales de los meses de prueba. `datos.loc[prueba, 'units_expected']`: lo que pronosticó DanuStore para esos meses.
- `fila['Modelo'] = ...`: le agrega datos al diccionario. `.append(fila)`: lo guarda en la lista.

Línea base (80/20)	WAPE	Qué significa
Pronóstico actual de DanuStore	468 %	Espera ~6 veces lo que se vende
Igual que el mes pasado	19.8 %	Método simple
Igual que el mismo mes del año pasado	0.29 %	Casi perfecto **porque los datos se repiten**

Sección 8 · Modelo 1: Random Forest

```
opciones_rf = {
    'n_estimators=100, max_depth=None, min_samples_leaf=1': RandomForestRegressor(n_estimators=100,
max_depth=None, min_samples_leaf=1, random_state=42),
    'n_estimators=200, max_depth=15, min_samples_leaf=2':   RandomForestRegressor(n_estimators=200,
max_depth=15, min_samples_leaf=2, random_state=42),
    'n_estimators=300, max_depth=8, min_samples_leaf=5':    RandomForestRegressor(n_estimators=300,
max_depth=8, min_samples_leaf=5, random_state=42),
}

for nombre, corte in particiones.items():
    entrenamiento = datos['fecha'] < corte
    prueba = datos['fecha'] >= corte

    for hiperparametros, modelo in opciones_rf.items():
        modelo.fit(X[entrenamiento], y[entrenamiento])     # entrenar
        prediccion = modelo.predict(X[prueba])               # pronosticar

        fila = evaluar(y[prueba], prediccion)                # calcular las métricas
        fila['Modelo'] = 'Random Forest'
        fila['Partición'] = nombre
        fila['Hiperparámetros'] = hiperparametros
        resultados.append(fila)
```

> **Cómo funciona Random Forest:** hace muchos árboles de decisión (preguntas tipo "¿promedio_3_meses > 200? → ¿es noviembre? → ¿es CDMX? → vende ~350"). Cada árbol aprende con una muestra distinta de los datos, y el pronóstico final es el **promedio de todos los árboles**.

- `opciones_rf` : diccionario con las 3 configuraciones. A la izquierda el nombre (texto) y a la derecha el modelo.
- **Hiperparámetros** (las "perillas" del modelo): `n_estimators` = número de árboles; `max_depth` = cuántas preguntas seguidas puede hacer cada árbol (None = sin límite); `min_samples_leaf` = mínimo de filas al final de cada rama (evita memorizar).
- `random_state=42` : fija el azar para que siempre salgan los mismos resultados.
- **Dos `for`, uno dentro del otro:** el de afuera recorre las 3 particiones; el de adentro, las 3 configuraciones → 3 × 3 = 9 modelos.
- `.fit(X, y)` = **entrenar** (aprende con pistas y respuestas). `.predict(X)` = **pronosticar** (solo con pistas).

Configuración (80/20)	WAPE
100 árboles, sin límite de profundidad	**5.98 %**
200 árboles, profundidad 15	7.44 %
300 árboles, profundidad 8	12.10 %

Sección 9 · Modelo 2: XGBoost

```
opciones_xgb = {
    'n_estimators=300, learning_rate=0.1, max_depth=6':  XGBRegressor(n_estimators=300,
learning_rate=0.1, max_depth=6, random_state=42),
    'n_estimators=500, learning_rate=0.05, max_depth=4': XGBRegressor(n_estimators=500,
learning_rate=0.05, max_depth=4, random_state=42),
    'n_estimators=800, learning_rate=0.03, max_depth=8': XGBRegressor(n_estimators=800,
learning_rate=0.03, max_depth=8, random_state=42),
}
... (los mismos dos for que en Random Forest)
```

> **Cómo funciona XGBoost:** también usa árboles, pero **uno tras otro**: el primero pronostica, el segundo aprende a corregir los errores del primero, el tercero corrige lo que quedó mal, y así. El pronóstico es la **suma de todas las correcciones**.

- Hiperparámetro nuevo: `learning_rate` = qué tanto corrige cada árbol. Más bajo = más lento pero más cuidadoso (por eso necesita más árboles).

Configuración (80/20)	WAPE
800 árboles, learning_rate 0.03, profundidad 8	**6.58 %**
300 árboles, learning_rate 0.1, profundidad 6	9.09 %
500 árboles, learning_rate 0.05, profundidad 4	11.79 %

Sección 10 · Comparación y validación

```
tabla_final = pd.DataFrame(resultados)[['Partición', 'Modelo', 'Hiperparámetros', 'MAE', 'RMSE', 'WAPE
(%)', 'R2']]

resumen = tabla_final.pivot_table(index='Modelo', columns='Partición', values='WAPE (%)', aggfunc='min')
```

`pd.DataFrame(resultados)` convierte la lista de filas en una tabla. `pivot_table(..., aggfunc='min')` deja el **menor error** de cada modelo en cada partición:

Modelo	80/20	70/30	60/40
Ingenuo estacional (año anterior)	0.29	0.31	0.30
Random Forest	**5.98**	**6.07**	**6.12**
XGBoost	6.58	6.97	7.05
Ingenuo (mes anterior)	19.83	18.65	18.17
Pronóstico actual	467.95	471.23	483.00

10.1 Validación (elegir sin "ver" la prueba)

```
entrenamiento_val = datos['fecha'] < '2025-05-01'
validacion = (datos['fecha'] >= '2025-05-01') & (datos['fecha'] < '2025-09-01')

for hiperparametros, modelo in opciones_rf.items():
    modelo.fit(X[entrenamiento_val], y[entrenamiento_val])
    fila = evaluar(y[validacion], modelo.predict(X[validacion]))
    ...
mejor_hiper_rf = tabla_validacion[tabla_validacion['Modelo'] == 'Random Forest']
['Hiperparámetros'].iloc[0]
```

```
Configuración elegida para Random Forest: n_estimators=100, max_depth=None, min_samples_leaf=1
Configuración elegida para XGBoost: n_estimators=800, learning_rate=0.03, max_depth=8
```

- Separa un "mini examen" dentro del entrenamiento: entrena hasta abril 2025 y evalúa con mayo–agosto 2025.
- El `&` significa "y": la fecha es mayor o igual a mayo **y** menor a septiembre.

- `.iloc[0]` toma la primera fila; como la tabla está ordenada por WAPE, esa es la mejor.
- Resultado: elige las mismas configuraciones (validación: RF 6.28 %, XGB 7.91 %).

> **Cómo decirlo:** "La validación confirmó las mismas configuraciones. Como los meses de prueba ya los había visto en el análisis, falta probar con datos nuevos."

Sección 11 · Resultados de los mejores modelos

```
mejor_rf = opciones_rf[mejor_hiper_rf]
mejor_rf.fit(X[entrenamiento], y[entrenamiento])

mejor_xgb = opciones_xgb[mejor_hiper_xgb]
mejor_xgb.fit(X[entrenamiento], y[entrenamiento])

resultado = datos.loc[prueba, ['fecha', 'region', 'units_sold', 'units_expected']].copy()
resultado['pronostico_rf'] = mejor_rf.predict(X[prueba])
resultado['pronostico_xgb'] = mejor_xgb.predict(X[prueba])
```

```
Random Forest: {'MAE': 13.31, 'RMSE': 20.8, 'WAPE (%)': 5.98, 'R2': 0.988}
XGBoost:       {'MAE': 14.64, 'RMSE': 20.54, 'WAPE (%)': 6.58, 'R2': 0.988}
```

- `opciones_rf[mejor_hiper_rf]`: saca del diccionario el modelo ganador usando su nombre.
- Se entrena con todo hasta agosto 2025 y se pronostica sep–dic 2025.
- `resultado`: tabla con lo real y lo pronosticado lado a lado.

Las gráficas de la sección 11

Gráfica	Qué muestra	Qué decir
Real vs. pronóstico por mes	Suma por mes de lo real, RF y XGB	"Los modelos siguen las ventas reales, incluida la subida de nov–dic; RF se desvía menos de 2 % por mes."
Real vs. estimado (puntos)	Cada punto es un producto-región-mes	"Los puntos quedan pegados a la línea roja = buen pronóstico."
Histograma de residuales	Distribución de (real − estimado)	"Centrado en 0: no sobrestima ni subestima. 90 % de los errores entre −30 y +33 unidades."
Residuales vs. estimado	Error contra el valor pronosticado	"No hay patrón; el error en unidades crece en los productos grandes, pero en % es menor."
Error por región	WAPE por región	"RF es parejo (5.2–8.2 %); XGB falla más en regiones chicas (hasta 12.7 %)."
Importancia de variables	Qué pistas usa más el modelo	"Promedio de 3 meses (~0.92) y mes (~0.03). Confirma la selección."

La función `graficar_residuales(nombre, pronostico)` hace las 3 gráficas de errores; se llama una vez para cada modelo.

Sección 12 · Impacto del preprocesamiento

```
from sklearn.base import clone

grupos = {
    '1. Básicas': basicas,
    '2. Básicas + precio': basicas + ['static_price'],
    '3. Básicas + ventas pasadas (modelo final)': basicas + rezagos,
    '4. Básicas + ventas pasadas + precio': basicas + rezagos + ['static_price'],
}

for nombre_modelo, modelo_elegido in [('Random Forest', mejor_rf), ('XGBoost', mejor_xgb)]:
    for nombre_grupo, columnas in grupos.items():
        X_grupo = pd.get_dummies(datos[columnas], columns=['region', 'category', 'subcategory'],
dtype=int)
        modelo = clone(modelo_elegido)
        modelo.fit(X_grupo[entrenamiento], y[entrenamiento])
        ...
```

- Se entrena el mismo modelo con 4 grupos de pistas diferentes para ver cuánto ayudó cada una.
- `clone()`: hace una copia del modelo **sin entrenar**, con los mismos hiperparámetros, para no "ensuciar" el modelo ganador.

Grupo de variables (Random Forest)	WAPE
1. Básicas (región, categoría, mes)	22.65 %
2. Básicas + precio	6.44 %
3. Básicas + ventas pasadas (**modelo final**)	**5.98 %**
4. Básicas + ventas pasadas + precio	5.87 %

> **Cómo decirlo:** "Crear las ventas pasadas bajó el error de 22.6 % a 6 %. Agregar el precio casi no cambia nada (5.87 % vs. 5.98 %), así que el modelo no depende de él."

Sección 13 · Pronóstico de varios meses (recursivo)

El problema: hasta aquí, para pronosticar octubre usamos las ventas reales de septiembre. Pero si DanuStore planea en agosto, todavía no sabe cuánto va a vender en septiembre.

```
def calcular_rezagos(tabla, columna):
    grupo = tabla.groupby(['product_id', 'region'])[columna]
    tabla['ventas_mes_anterior'] = grupo.shift(1)
    ... (lo mismo que la sección 3)
    return tabla

meses_prueba = ['2025-09-01', '2025-10-01', '2025-11-01', '2025-12-01']

for nombre_modelo, modelo in [('Random Forest', mejor_rf), ('XGBoost', mejor_xgb)]:

    # 1) "Borramos" las ventas desde septiembre (como si aún no las conociéramos)
    historia = df.copy()
    historia['ventas_simuladas'] = historia['units_sold']
    historia.loc[historia['fecha'] >= '2025-09-01', 'ventas_simuladas'] = np.nan

    # 2) Pronosticamos un mes a la vez y guardamos el pronóstico como si fuera la venta real
    for mes in meses_prueba:
        historia = calcular_rezagos(historia, 'ventas_simuladas')
        X_historia = pd.get_dummies(historia[variables], columns=['region', 'category', 'subcategory'],
dtype=int)
        filas_del_mes = historia['fecha'] == mes
        historia.loc[filas_del_mes, 'ventas_simuladas'] = modelo.predict(X_historia[filas_del_mes])

    # 3) y 4) Comparamos contra las ventas reales, mes por mes y en bloque
```

Paso	Qué pasa
1	Copia los datos y pone `np.nan` (vacío) en las ventas de sep–dic: "no las conozco".
2 – septiembre	Calcula las ventas pasadas (jun, jul, ago son reales) → pronostica septiembre → guarda ese pronóstico en la columna.
2 – octubre	Ahora "septiembre" es **el pronóstico**, no lo real → pronostica octubre → lo guarda.
2 – nov y dic	Igual: cada mes usa pronósticos anteriores, así que los errores se van acumulando.
3 y 4	Compara lo pronosticado contra lo real.

Mes	Meses adelante	Random Forest	XGBoost
Septiembre	1	5.83 %	7.28 %
Octubre	2	9.42 %	9.52 %
Noviembre	3	10.94 %	11.00 %
Diciembre	4	**12.26 %**	12.03 %
Bloque sep–dic	1 a 4	**9.91 %**	10.19 %

> **Cómo decirlo:** "Si planeo 4 meses desde agosto, el error del bloque es 9.9 %, y en diciembre, que está 4 meses adelante, llega a 12.3 %. Sigue siendo muchísimo mejor que el 468 % del pronóstico actual."

Sección 14 · Conclusiones (lo que tienes que decir)

- **Mejor modelo de ML:** Random Forest (100 árboles, sin límite). Gana en las 3 particiones (5.98 / 6.07 / 6.12 %) y la validación lo confirmó. XGBoost queda cerca, pero es menos parejo entre regiones.
- **Particiones:** resultados muy parecidos; como evalúan meses distintos, no digo que una sea mejor, solo que el modelo es estable.
- **Contra DanuStore:** ~6 % a un mes y 9.9 % en el bloque de 4 meses, contra 468 % del pronóstico actual.
- **Ojo:** copiar el año anterior da 0.3 % porque en estos datos 2025 repite a 2024. Es el método con menor error entre los evaluados; lo reporto como hallazgo de calidad de datos.
- **Preprocesamiento:** las ventas pasadas bajaron el error de 22.6 % a 6 %; quitar el precio casi no cambia nada.
- **Hiperparámetros:** las configuraciones más limitadas tuvieron más error (RF de 6 % a 12 %; XGB de 6.6 % a 11.8 %). Se compararon configuraciones completas.

- **Beneficio para DanuStore:** conoce cómo se comportan sus productos (temporada, categorías, regiones) y puede planear el inventario con base en ventas reales.

> **Limitantes:** (1) 2025 repite a 2024, así que los errores son optimistas; (2) solo ventas de clientes registrados; (3) el error crece con el horizonte; (4) no sirve para productos nuevos sin 3 meses de historial; (5) los meses de prueba se vieron durante la exploración, así que falta probar con datos nuevos.

Glosario rápido de Python

Código	Significa
`df['columna']`	Tomar una columna de la tabla
`df[condición]`	Quedarse solo con las filas que cumplen la condición
`df.loc[filas, 'col']`	Tomar (o cambiar) ciertas filas de una columna
`groupby(...).sum()`	Agrupar y sumar (como una tabla dinámica de Excel)
`pivot_table(...)`	Tabla dinámica: filas, columnas y valores
`shift(n)`	Recorrer la columna n renglones hacia abajo
`{ 'a': 1 }`	Diccionario: nombre → valor
`[ ]` y `.append()`	Lista vacía y agregarle un elemento
`for x in ...:`	Repetir el bloque para cada elemento
`def nombre(...):`	Crear una función (receta reutilizable)
`.fit(X, y)` / `.predict(X)`	Entrenar el modelo / pronosticar
`np.nan`	Valor vacío
`&` / `~`	"y" / "no" en condiciones